

BAX 423 — Homework 1 — Part 1

Logan Garcia, Bonnie Hines

1. Frontier (closed) model vs open source for a song recommender

If you pay for a top vendor model, you’re basically buying speed and fewer headaches: someone else hosts it, patches it, and deals with a lot of the safety and content-policy stuff. For a small team that matters—you’re not babysitting GPUs and giant checkpoints all day. The downside is cost scales with traffic, you’re stuck on their roadmap, and if they change the model your embeddings and offline jobs can all move at once. You also have to be comfortable sending user data out of your stack unless you negotiate something tighter.

Open source is the opposite trade. You host it, fine-tune it, and own the behavior end to end. That’s better when you care about cost at scale, weird domain-specific music data, or compliance that says listening history can’t leave your VPC. The catch is you need people who can actually run and update that stack, and you’re on the hook when something breaks or the model ages.

2. Features from listening history, and what needs to be “streaming” at serve time

Stuff you’d actually build from logs: how often someone plays an artist or genre, recency-weighted play counts, skip vs complete, time-of-day patterns, maybe a short recent sequence of track IDs for “what they’ve been playing this week.” You might also have coarse location or device if the product uses it.

For a playlist that refreshes once a day or week, most of that can be batch: nightly or hourly jobs that write tables or features to a store, and the app just reads precomputed results. You don’t need those features updating every second.

If you’re recommending the next track off what’s playing right now, you care about what’s happening in this session—current track, last few skips or completes, maybe queue—so those pieces need to be available with very low latency. That’s where you end up with real-time or near-real-time pipes. The daily playlist path is heavier offline work and caching; the in-session path is tighter SLAs and more requests per user.

3. Daily “For you” playlist vs a recommendation for each new track

The batch playlist case is forgiving on latency. You can run bigger models, score more candidates, and ship a list users see hours later. Failures are easier to hide—you retry the job, you don’t block a button press.

Per-track recommendations fire all the time while someone is listening. You need fast inference, a small set of features you can fetch quickly, and usually a simpler model or aggressive caching so you don't melt the cluster. You also care more about jarring jumps (genre whiplash) because the user notices immediately. So the engineering work shifts from big overnight pipelines to online serving, feature stores, and rate limits—not the same problem as a once-a-day job.

4. When an artist or subgenre blows up overnight

First you watch the product metrics: clicks, skips, completion, broken out for new tracks and new tags—not just one global accuracy number. If TikTok drives a spike in a sound, your old collaborative model might be stale before the nightly train finishes.

In practice that means shorter refresh windows for the data you train on, favoring recent listens when you retrain, and having a path for cold items (audio or text features, or “similar to” clusters) so new stuff isn't invisible. Some teams ship model updates gradually (small % of users first) so you don't tank everyone at once. Exploration matters too—if the system only exploits old co-occurrence, it never learns the new thing until it's already huge. Retraining on a schedule only works if someone is actually looking at whether new slices of the catalog are doing okay.